Министерствонауки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе№1

«Арифметические операции с полиномами»

Выполнил:
студент группы 3824Б1ФИ2
Щербаков Н.А.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 8
3.1. Описание структуры программы 8
3.2. Описание структур данных 8
3.3. Описание алгоритмов 13
4. Результаты экспериментов и тестирования 16
Заключение 18
Литература 19

Введение

Полиномы являются одной из основных математических моделей, широко применяемых в различных областях науки и техники, таких как численные методы, моделирование процессов, компьютерная графика и системы компьютерной алгебры. Полиномы от нескольких переменных позволяют описывать зависимости между несколькими величинами и представлять сложные математические выражения в удобной форме.

В программировании важную роль играет эффективное представление полиномов в памяти и реализация операций над ними. Полином можно рассматривать как набор мономов, каждый из которых содержит коэффициент и степени переменных. При выполнении арифметических операций необходимо корректно обрабатывать подобные мономы и исключать элементы с нулевыми коэффициентами.

В данной работе хранение полинома реализовано в виде односвязного списка с фиктивной головой, элементы которого представляют мономы с ненулевыми коэффициентами. Каждый элемент списка содержит коэффициент монома и степень переменных, представленную в свернутом виде. Использование односвязного списка позволяет эффективно хранить полиномы произвольного размера и выполнять операции добавления и обработки мономов.

1. Постановка задачи

Целью данной работы является разработка программы на языке C++, выполняющая арифметические операции с полиномами от трех переменных: X, Y, Z.

Реализуемые операции:

1. Сложение полиномов;
2. Вычитание полиномов;
3. Умножение полинома на вещественную константу;
4. Умножение двух полиномов.

Особенности реализации:

1. **Структура хранения:** Хранения полинома должно быть представлено в виде списка мономов с ненулевыми вещественными коэффициентами. Элементы списка хранить упорядоченными. В данной работе используется односвязный список с фиктивной головой.
2. **Формат хранения степеней:** Степень полинома должна храниться в «свернутом» виде как трехзначное целое число: число сотен – степень при переменной X, число десятков – степень при переменной Y, число единиц – степень при переменной Z.
3. **Алгоритм сложения:** Сложение полиномов должно осуществляться алгоритмом слияния упорядоченных массивов.
4. **Алгоритм вычитания:** Вычитание полиномов осуществляется через сложение полиномов с умножением второго полинома на константу (-1).
5. **Контроль хранения:** При умножении и сложении (вычитании) в итоговом полиноме должны быть приведены подобные слагаемые и не должны храниться мономы с нулевым коэффициентом.
6. **Контроль степеней:** Если при умножении полиномов полученные степени переменных больше 9, то программа должна выводить сообщение об ошибке.
7. **Ввод данных пользователем:** Программа должна корректно обрабатывать пользовательский ввод полинома, даже если пользователь ввёл полином, не упорядочив в нём мономы.
8. **Тестирование программы:** Работоспособность программы должна быть проверена с помощью фреймворка Google Test.
9. **Ограничение степеней:** Полином должен быть составлен из мономов с ограничением на степень каждой переменной X, Y, Z от 0 до 9.
10. **Консольное приложение:** Программа должна иметь пользовательский интерфейс (консольное приложение).

2. Руководство пользователя

Данная программа предназначена для работы с полиномами трёх переменных X, Y и Z. Пользователь может создавать полиномы, выводить их на экран, а также выполнять над ними арифметические операции, вводя соответствующую команду.

2.1 Запуск программы

Программа представляет собой консольное приложение с меню. После запуска отображается список доступных команд:

```
+-------------------------MENU-------------------------+
1 - Creating/Replacing the first polynomial
2 - Creating/Replacing the second polynomial
3 - Outputting the first polynomial to the console
4 - Outputting the second polynomial to the console
5 - Add polynomials
6 - Subtract polynomials
7 - Multiply a polynomial by a constant
8 - Multiply polynomials
9 - Outputting the program menu to the console
0 - Shutting down the program
+------------------------------------------------------+
+--------------------+ENTER_CHOICE+--------------------+
```

Рис. 1. Меню программы

2.2 Ввод полинома (команды 1 и 2)

При выборе команд 1 или 2 пользователь вводит соответственно первый или второй полином. Полином состоит из набора мономов, которые пользователь вводит постепенно.

Для каждого монома необходимо ввести:

- Коэффициент – вещественное число (не равное нулю).
- Степень монома – целое число от 0 до 999.

Степень монома задаётся в виде трёхзначного числа, где:

- Сотни – степень переменной X.
- Десятки – степень переменной Y.
- Единицы – степень переменной Z.

Пример: Если пользователь вводит Coefficient: 5 и Degree: 231, то это соответствует моному: 5x^2y^3z^1.

После ввода монома программа предлагает:

- 1 - Continue entering the polynomial (Продолжить ввод полинома со следующего монома)
- 0 - Finish entering the polynomial (Завершить ввод полинома).

После завершения ввода полинома программа автоматически:

- Удаляет мономы с нулевыми коэффициентами.
- Сортирует мономы по степеням.
- Объединяет мономы с одинаковыми степенями.

2.3 Вывод полинома (команды 3 и 4)

При выборе команд 3 или 4 программа выводит соответственно первый или второй полином в консоль.

Если соответствующий полином не был введён пользователем, то программа сообщит о том, что **полином пуст**: The polynomial is empty.

2.4 Операции с полиномами (команды 5, 6, 7 и 8)

1. При выборе команды 5 программа выполнит **операцию сложения** двух полиномов и выведет результат в консоль.
   Если какой-либо из полиномов не был введён пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

2. При выборе команды 6 программа начнёт выполнение **операции вычитания. Пользователю будет предложено:**
   - 0 - subtract the second from the first polynomial (p1 - p2). То есть вычесть второй полином из первого.
   - 1 - subtract the first from the second polynomial (p2 - p1). То есть вычесть первый полином из второго.

   После выбора пользователя результат вычитания будет выведен в консоль.
   Если какой-либо из полиномов не был введён пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

3. При выборе команды 7 программа начнёт выполнение **операции умножения полинома на вещественную константу**.
   Пользователю будет предложено:
   - 0 - multiply the first(p1) polynomial by a constant. То есть умножить первый полином на константу.
   - 1 - multiply the second(p2) polynomial by a constant. То есть умножить второй полином на константу.

   После выбора пользователю будет предложено ввести вещественную константу. Результат умножения на константу будет выведен в консоль.
   Если соответствующий полином не был введён пользователем, то программа сообщит о том, что **полином пуст**.

4. При выборе команды 8 программа выполнит **операцию умножения** двух полиномов и выведет результат в консоль.
   Если какой-либо из полиномов не был введён пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

2.5 Вывод меню программы (команда 9)

При выборе команды 9 программы продублирует (выведет) своё меню в консоль.

2.6 Завершение работы программы (команда 0)

При выборе команды 0 программа завершится.

2.7 Обработка ошибок

Программа учитывает некорректный ввод пользователя и предоставляет ему возможность повторить соответствующий ввод без завершения программы:

- При некорректном вводе программа выведет: «INVALID INPUT. ‘повтор требований ввода от пользователя’».
- При умножении двух полиномов и выходе за пределы по степеням программа выведет: «The degrees of the obtained variables are greater than 9. The maximum degree of a polynomial is 999.».

2.8 Пример использования программы:

```
+--------------------+ENTER_CHOICE+--------------------+
1
The coefficients of monomials are real numbers.
The degree of a monomial is a three-digit number, where the hundreds digit is the d
egree of variable X, the tens digit is the degree of variable Y, and the units digi
t is the degree of variable Z.
Enter the coefficient of the monomial:
3
Enter the degree of the monomial:
234
1 - Continue entering the polynomial, 0 - Finish entering the polynomial.
Enter:
1
Enter the coefficient of the monomial:
5
Enter the degree of the monomial:
351
1 - Continue entering the polynomial, 0 - Finish entering the polynomial.
Enter:
0
+--------------------+ENTER_CHOICE+--------------------+
3
5x^3y^5z^1 +3x^2y^3z^4
```

Рис. 2. Ввод первого полинома

```
+--------------------+ENTER_CHOICE+--------------------+
2
The coefficients of monomials are real numbers.
The degree of a monomial is a three-digit number, where the hundreds digit is the d
egree of variable X, the tens digit is the degree of variable Y, and the units digi
t is the degree of variable Z.
Enter the coefficient of the monomial:
-4.4
Enter the degree of the monomial:
222
1 - Continue entering the polynomial, 0 - Finish entering the polynomial.
Enter:
0
+--------------------+ENTER_CHOICE+--------------------+
4
-4.4x^2y^2z^2
```

Рис. 3. Ввод второго полинома

```
+--------------------+ENTER_CHOICE+--------------------+
5
5x^3y^5z^1 +3x^2y^3z^4 -4.4x^2y^2z^2
+--------------------+ENTER_CHOICE+--------------------+
7
0 - multiply the first(p1) polynomial by a constant, 1 - multiply the second(p2) po
lynomial by a constant.
Enter:
0
Enter a constant (real number):
-8.3
-41.5x^3y^5z^1 -24.9x^2y^3z^4
+--------------------+ENTER_CHOICE+--------------------+
8
-22x^5y^7z^3 -13.2x^4y^5z^6
+--------------------+ENTER_CHOICE+--------------------+
```

Рис. 4. Сложение, умножение на константу, умножение

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из двух проектов (Lab1Code и Lab1Tests), построена по модульному принципу и состоит из следующих компонентов:

- **Polynom.h**: файл, содержащий объявления и реализации классов Forward_list_with_f_head (односвязный список с фиктивной головой) и Polynomial (полином), а также вспомогательные структуры Monom (моном) и Node (элемент списка). Здесь находится математическая логика работы программы.
- **Console_app.cpp**: файл, содержащий код консольного приложения. Здесь находится логика управления программой, обработка некорректных случаев, ошибок.
  Функции:
  - **main()** – главная фукнция, реализующую работу с пользователем.
  - **input_polynom()** – функция реализующая ввод полинома пользователем с консоли.
  - **print_menu()** – функция для вывода меню программы в консоль.
- **Test.cpp:** файл содержащий все тесты Google Test методов классов Forward_list_with_f_head и Polynomial, проверяющих корректность выполнения данной программы.

3.2. Описание структур данных

Основными структурами данных данной программы являются два класса **Forward_list_with_f_head** и **Polynomial**, а также вспомогательные структуры **Monom** и **Node**. Кроме того, в программе присутствует глобальная переменная **const double EPSILON**, которая нужна для корректного сравнения коэффициентов мономов с нулём.

1) <u>Структура Monom:</u>

Структура Monom предназначена для хранения одного монома полинома.

<u>Поля структуры:</u>

- **double** coeff – вещественный коэффициент монома.
- **size_t** deg – степень монома, представленная трёхзначным числом.

<u>Формат хранения степени:</u>

- **Единицы** – степень при переменной «z».
- **Десятки** – степень при переменной «y».
- **Сотни** – степень при переменной «x».

<u>Методы структуры:</u>

- Monom(**double** coefficient = 0.0, **size_t** degree = 0) : coeff{ coefficient }, deg{ degree } {}

<u>Функционал:</u> Конструктор по умолчанию и с параметрами. Инициализирует коэффициент и степень монома.

<u>Параметры:</u> coefficient – коэффициент монома (если не введён, то инициализируется нулём); degree – степень монома (если не введена, то инициализируется нулём).

2) ***<u>Структура Node:</u>***

Структура Node используется для представления элемента односвязного списка.

<u>Поля структуры:</u>

- **Monom** data – данные монома (коэффициент и степень).
- **Node*** next – указатель на следующий элемент списка.

<u>Методы структуры:</u>

- Node(**const Monom&** data_, **Node*** next_ = nullptr) : data{ data_ }, next{ next_ } {}

<u>Функционал:</u> Конструктор по умолчанию и с параметрами. Создаёт новый элемент односвязного списка.

<u>Параметры:</u> data_ – данные монома; next_ – указатель на следующий элемент списка (если не введён, то инициализируется nullptr).

3) ***<u>Класс Forward_list_with_f_head:</u>***

Класс Forward_list_with_f_head реализует односвязный список с фиктивной головой, предназначенный для хранения мономов полинома. Класс содержит методы добавления нового монома в список, сортировки мономов, проверки на пустоту списка. Фиктивная голова используется для упрощения операции вставки элементов.

<u>Поля класса:</u>

- **size_t** size – размер списка (количество элементов/мономов в списке).
- **Node*** head – указатель на фиктивную голову списка.

<u>Методы класса:</u>

- **Node*** find_middle_elem(**Node*** node_)

<u>Функционал:</u> Приватный метод. Нахождение среднего элемента списка, используется в алгоритме сортировки слиянием. Разрывает связь списка по указателям по найденному среднему элементу.

<u>Возвращаемое значение:</u> Node*, указатель на найденный средний элемент списка.

<u>Параметры:</u> node_ – указатель на первый элемент списка после фиктивной головы.

<u>Сложность:</u> O(n) – так как цикл выполняется примерно n/2 раз (из-за «быстрого» указателя fast), где n – число элементов в списке: O(n/2) = O(n).

- **Node*** merge(**Node*** left, **Node*** right)

<u>Функционал:</u> Приватный метод. Слияние двух отсортированных списков, используется в алгоритме сортировки слиянием. Рекурсивно вызывается метод merge для пары элементов двух отсортированных списков, сравниваются их степени, выбирается элемент с большей степенью. Объединяет подобные мономы (элементы с одинаковой степенью). Проверяет мономы на нулевой коэффициент.

<u>Возвращаемое значение:</u> Node*, указатель на первый элемент после фиктивной головы нового списка после слияния двух отсортированных списков.

<u>Параметры:</u> left – указатель на левую часть списка (первый список); right – указатель на правую часть списка (второй спислок).

Сложность: O(n+m) – так как каждый элемент из двух списков обрабатывается ровно 1 раз, где n – число элементов в первом(left) списке, а m – число элементов во втором(right) списке.

- **Node*** merge_sort(**Node*** node_)

Функционал: Приватный метод. Рекурсивная функция для сортировки списка с помощью алгоритма сортировки слиянием. Разделяет список на две части, сортирует каждую часть рекурсивно и затем сливает отсортированные части обратно в один отсортированный список.

Возвращаемое значение: Node*, указатель на первый элемент после фиктивной головы отсортированного списка.

Параметры: node_ – указатель на первый элемент после фиктивной головы не отсортированного списка.

Сложность: O(n*log(n)) – сортировка слиянием.

- Forward_list_with_f_head()

Функционал: Конструктор по умолчанию. Создаёт пустой список и инициализирует фиктивную голову списка.

- Forward_list_with_f_head(**const Forward_list_with_f_head&** other_list)

Функционал: Конструктор копирования. Создаёт копию списка other_list путём последовательного копирования его элементов.

Параметры: other_list – копируемый список.

Сложность: O(n) – так как в цикле проходимся по всем n элементам копируемого списка other_list.

- **void** clear()

Функционал: Метод очистки списка. Удаляет все элементы списка, кроме фиктивной головы.

Сложность: O(n) – так как в цикле проходимся по всем n элементам списка, удаляя их.

- ~Forward_list_with_f_head()

Функционал: Деструктор. Вызывается метод clear() и удаляется фиктивная голова.

Сложность: O(n) – так как метод clear() имеет сложность O(n).

- **Forward_list_with_f_head&** operator=(**const Forward_list_with_f_head&** other_list)

Функционал: Перегруженный оператор присваивания. Создаёт копию односвязного списка other_list. Присутствует проверка на самоприсваивание.

Возвращаемое значение: Forward_list_with_f_head&, ссылка на текущий список.

Параметры: other_list – копируемый список.

Сложность: O(n) – так как вызывается конструктор копирования класса Forward_list_with_f_head, который имеет сложность O(n).

- **bool** is_empty() **const**

Функционал: Метод для проверки списка на пустоту.

Возвращаемое значение: bool, true – если список пуст, false – если список не пуст.

- **void** sort()

Функционал: Метод для сортировки списка с помощью алгоритма сортировки слиянием. Объединяем мономы с одинаковой степенью и удаляет мономы с нулевыми коэффициентами.

Сложность: O(n*log(n)) – так как вызывается приватный метод merge_sort(), который имеет сложность O(n*log(n)).

- **void** push_front(**double** coeff_, **size_t** deg_)

Функционал: Приватный метод добавления нового элемента в начало списка (после фиктивной головы). Присутствует проверка на нулевой коэффициент монома и выход значения степени монома за допустимые пределы. Нет сортировки элементов списка.

Параметры: coeff_ – вещественное значение коэффициента монома, deg_ – беззнаковое целое значение степени монома.

- **void** push_quick(**double** coeff_, **size_t** deg_)

Функционал: Метод добавления нового элемента в конкретное место списка, в зависимости от степени deg_. Присутствует проверка на нулевой коэффициент монома и выход значения степени монома за допустимые пределы. Благодаря поиску места вставки, список остаётся упорядоченным, отсортированным.

Параметры: coeff_ – вещественное значение коэффициента монома, deg_ – беззнаковое целое значение степени монома.

Сложность: O(n) – так как в цикле проходимся по всем элементам списка, чтобы найти место вставки (n – количество элементов списка).

- **std::vector<std::pair<double, size_t>>** get_list() **const**

Функционал: Метод, который нужен для тестирования класса, который возвращает данные списка (все элементы/мономы в виде вектора пар значений).

Возвращаемое значение: std::vector<std::pair<double, size_t>>, вектор пар значений (коэффициент/степень монома).

Сложность: O(n) – так как в цикле проходимся по всему списку, чтобы получить значения его элементов (n – количество элементов/мономов).

4) Класс Polynomial:

Класс Polynomial для представления полинома. Класс содержит методы сложения полиномов, вычитания полиномов, умножения полинома на вещественную константу, умножения полиномов, проверки полинома на пустоту, вывод полинома в консоль. Мономы полинома хранятся в упорядоченном виде (нет подобных мономов, нет мономов с нулевым коэффициентом, мономы выстроены в порядке убывания степени).

Поля класса:

- **Forward_list_with_f_head** polynom – односвязный список с фиктивной головой для хранения мономов полинома.

Методы класса:

- Polynomial()

Функционал: Конструктор по умолчанию. Вызывается конструктор по умолчанию класса Forward_list_with_f_head. Создаётся пустой полином.

- Polynomial(**const std::vector<std::pair<double, size_t>>&** source)

Функционал: Конструктор с параметром. Создаётся полином с данными из вектора пар значений source. Есть проверка на нулевой коэффициент.

Параметры: source – вектор пар значений. Содержит данные мономов полинома.

Сложность: O(n*log(n)) – так как в цикле проходимся по всем парам значений из source (n – число пар значений вектора source) и затем производим сортировку методом sort() класса Forward_list_with_f_head, который имеет сложность O(n*log(n)).

- Polynomial(**const Polynomial&** other_polynom)

Функционал: Конструктор копирования. Вызывается конструктор копирования класса Forward_list_with_f_head.

Параметры: other_polynom – копируемый полином.

Сложность: O(n) – так как конструктор копирования класса Forward_list_with_f_head имеет данную сложность.

- ~Polynomial()

Функционал: Деструктор. Вызывается деструктор класса Forward_list_with_f_head. Очищает полином.

Сложность: O(n) – так как деструктор класса Forward_list_with_f_head имеет данную сложность.

- **Polynomial&** operator=(**const Polynomial&** other_polynom)

Функционал: Перегруженный оператор присваивания. Создаёт копию полинома other_polynom. Присутствует проверка на самоприсваивание. Вызывается перегруженный оператор присваивания класса Forward_list_with_f_head.

Возвращаемое значение: Polynomial&, ссылка на текущий полином.

Параметры: other_polynom – копируемый полином.

Сложность: O(n) – так как перегруженный оператор присваивания класса Forward_list_with_f_head имеет данную сложность.

- **Polynomial** operator+(**const Polynomial&** other_polynom) **const**

Функционал: Перегруженный оператор сложения полиномов. Используется алгоритм слияния упорядоченных массивов. Объединяет подобные мономы. Присутствует проверка на нулевой коэффициент.

Возвращаемое значение: Polynomial, возвращает новый полином – результат сложения двух полиномов.

Параметры: other_polynom – второй полином для сложения.

Сложность: O(n+m) – так как в циклах проходимся по всем мономам двух полиномов (n – количество мономов в текущем полиноме; m – количество мономов в полиноме other_polynom).

- **Polynomial** operator*(**const double&** val) **const**

Функционал: Перегруженный оператор умножения текущего полинома на вещественную константу val. Присутствует проверка на нулевой коэффициент.

Возвращаемое значение: Polynomial, возвращает новый полином – результат умножения текущего полинома на вещественную константу val.

Параметры: val – вещественная константа.

Сложность: O(n) – так как в цикле проходимся по всем мономам текущего полинома (n – количество мономов в текущем полиноме).

- **Polynomial** operator-(**const Polynomial&** other_polynom) **const**

Функционал: Перегруженный оператор вычитания полиномов. Применяются перегруженный оператор сложения полиномов и перегруженный оператор умножения полинома на вещественную константу (в данном случае на (-1)).

Возвращаемое значение: Polynomial, возвращает новый полином – результат вычитания двух полиномов.

Параметры: other_polynom – второй полином для вычитания.

Сложность: O(n+m) – так как перегруженный оператор умножения полинома на вещественную константу имеет сложность O(n), а перегруженный оператор сложения полиномов имеет сложность O(n+m).

- **Polynomial** operator*(**const Polynomial&** other_polynom) **const**

Функционал: Перегруженный оператор умножения полиномов. Присутствует проверка на нулевые коэффициенты мономов. Присутствует проверка на выход значения степени монома за пределы допустимых значений.

Возвращаемое значение: Polynomial, возвращает новый полином – результат умножения двух полиномов.

Параметры: other_polynom – второй полином для умножения.

Сложность: O(n*m) – так как в циклах каждый моном текущего полинома умножаем на каждый моном полинома other_polynom (n – количество мономов в текущем полиноме; m – количество мономов в полиноме other_polynom).

- **void** print_polynom() **const**

Функционал: Метод для вывода полинома в консоль. Нужен для реализации консольного приложения.

Сложность: O(n) – так как в цикле проходимся по всем мономам текущего полинома (n – количество мономов в текущем полиноме).

- **bool** is_empty()

Функционал: Метод для проверки полинома на пустоту.

Возвращаемое значение: bool, true – если полином пуст, false – если полином не пуст.

- **std::vector<std::pair<double, size_t>>** get_polynom() **const**

Функционал: Метод, который нужен для тестирования класса, который возвращает данные полинома (все элементы/мономы в виде вектора пар значений).

Возвращаемое значение: std::vector<std::pair<double, size_t>>, вектор пар значений (коэффициент/степень монома).

Сложность: O(n) – так как в цикле проходимся по всему полиному, чтобы получить значения его мономов (n – количество мономов в полиноме).

3.3. Описание алгоритмов

1) **Алгоритм сортировки мономов (сортировка слиянием для списка).**

Для поддержания корректного математического порядка мономов используется алгоритм сортировки слиянием **merge_sort()** на односвязном списке с фиктивной головой.

- Список рекурсивно делится на две части через метод **find_middle_elem()**, который находит средний элемент, разрывая связь между левым и правым подсписком.
- Метод **merge()** объединяет два отсортированных подсписка:
  - Если степени текущих мономов различны, то моном с большей степенью добавляется первым в объединённый список.
  - Если степени равны, то коэффициенты складываются. Если результат близок к нулю (меньше **EPSILON**), то моном удаляется.
- После сортировки обновляется размер списка (в методе **sort()**).

Этот подход обеспечивает корректное упорядочивание мономов по степеням и удаление подобных мономов. Алгоритм работает за **O(n*log(n))**.

2) **Алгоритм добавления мономов (push_quick()).**

Для вставки монома в отсортированный список:

- Происходит поиск позиции вставки с помощью прохода по списку от фиктивной головы.
- Если найден моном с такой же степенью, то коэффициенты мономов складываются. Если результат равен нулю, то моном удаляется, обновляются связи между мономами и обновляется размер списка.
- Если равной степени нет, то создаётся новый моном и вставляется на найденное место, чтобы поддерживать сортировку.

Алгоритм работает за **O(n)** в худшем случае, где n – количество мономов в списке.

3) **Алгоритм сложения полиномов (Алгоритм слияния упорядоченных массивов, operator+(const Polynomial& other_polynom)) .**

Сложение полиномов реализовано через одновременный проход по обоим отсортированным спискам:

- Если степени текущих мономов различны, моном с большей степенью добавляется первым в результирующий полином.
- Если степени равны, то коэффициенты мономов складываются. Нулевые коэффициенты, как результат сложения, игнорируются.
- После завершения обхода одного из полиномов оставшиеся мономы другого полинома добавляются в результирующий полином.
- В конце вызывается сортировка методом **sort()**, чтобы гарантировать корректное упорядочивание и удаление возможных нулевых мономов.

Алгоритм работает за **O(n+m)**, где n и m – размеры полиномов.

4) **Алгоритм умножения полиномов (operator*(const Polynomial& other_polynom)) .**

Умножение выполняется по схеме «каждый моном первого полинома на каждый моном второго»:

- Разряды степеней (сотни, десятки, единицы) каждого монома суммируются отдельно.
- Если сумма разрядов превышает 9 для какой-либо переменной, то выбрасывается исключение **std::range_error** (ограничение степени 999).

- Коэффициенты перемножаются, проверяются на близость к нулю (меньше **EPSILON**).
- Новый моном вставляется в результирующий полином через **push_front()**.
- После завершения всех итераций вызывается метод **sort()**, чтобы удалить нулевые мономы и упорядочить результат.

Алгоритм работает за O(n*m), где n и m – размеры полиномов.

5) **Удаление нулевых коэффициентов.**

Удаление нулевых коэффициентов мономов встроено в сортировку слиянием **sort()** и во вставку **push_quick()**.

- При суммировании или вставке мономов проверяется условие **std::fabs(coeff) < EPSILON** (Близость коэффициента к нулю).
- Если коэффициент близок к нулю, то моном/элемент удаляется из списка, а размер списка уменьшается.

Это обеспечивает корректное представление полинома без лишних элементов.

4. Результаты экспериментов и тестирования

Для проверки корректности работы программы проведено комплексное модульное тестирование с использованием **Google Test**. Тесты охватывают ключевые функции классов **Forward_list_with_f_head** и **Polynomial**, обеспечивая проверку корректности структур данных, алгоритмов вставки, сортировки, арифметических операций и обработки исключений.

Тестовое покрытие включает следующие логические группы:

1) **Тестирование класса Forward_list_with_f_head.**
   - Создание пустого списка:
     Тест **CreateDefaultEmptyListIsCorrectNoThrow** подтвердил, что после вызова конструктора по умолчанию список корректно инициализируется пустым, а метод **is_empty()** возвращает **true**.
   - Вставка мономов push_quick():
     Тест **PushQuickIsCorrectNoThrow** показал, что вставка мономов в произвольном порядке сохраняет правильный порядок по степеням. Вставка новых элементов с меньшей или большей степенью корректно помещает их в начало или конец списка. Проверка через **get_list()** подтвердила соответствие коэффициентов и степеней ожидаемым значениям.
   - Копирование и присваивание:
     Тесты **CreateListByCopyingIsCorrectNoThrow** и **AssignmentIsCorrectNoThrow** подтвердили корректность реализации глубокого копирования. Изменение копии списка не влияет на оригинальный список. Проверка через **get_list()** подтвердила соответствие коэффициентов и степеней ожидаемым значениям.
   - Обработка некорректных данных:
     Тест **PushFrontAnyThrow** показал генерацию исключений при попытке вставки мономов с нулевым коэффициентом или со степенью вне диапазона [0; 999], что обеспечивает контроль корректности данных.
2) **Тестирование класса Polynomial.**
   - Создание полиномов:
     Тесты **CreateDefaultEmptyPolynomIsCorrectNoThrow** и **CreateNotEmptyPolynomIsCorrectNoThrow** подтвердили корректное создание пустых и непустых полиномов. Мономы с нулевым коэффициентом автоматически игнорируются при формировании полинома. Метод **is_empty()** возвращает ожидаемое значение. Проверка через **get_polynom()** подтвердила соответствие коэффициентов и степеней ожидаемым значениям.
   - Обработка некорректных данных:
     Тест **CreateNotEmptyPolynomIsCorrectAnyThrow** подтвердил генерацию исключений при попытке передать полиному мономы со степенью вне диапазона [0; 999], что обеспечивает контроль корректности данных.
   - Копирование и присваивание:

Тест **AssignmentIsCorrectNoThrow** показал, что присваивание одного полинома другому корректно копирует все мономы без влияния на исходный объект.

- Сложение и вычитание полиномов:
  Тест **PolynomAdditionAndSubtractionIsCorrectNoThrow** проверил корректность сложения и вычитания полиномов с разными степенями. Результат подтверждает правильное объединение мономов с одинаковыми степенями и сохранение порядка по убыванию степеней. Проверка через **get_polynom()** подтвердила соответствие коэффициентов и степеней ожидаемым значениям.
- Умножение на константу:
  Тесты **PolynomMultiplicationConstantIsCorrectNoThrow** и **PolynomMultiplicationZeroNoThrow** продемонстрировали корректное масштабирование коэффициентов полинома и удаление мономов с нулевым коэффициентом. Проверка через **get_polynom()** подтвердила соответствие коэффициентов и степеней ожидаемым значениям.
- Умножение полиномов:
  Тест **MultiplicationPolynomIsCorrectNoThrow** подтвердил реализацию свойства умножения «каждый моном первого полинома на каждый моном второго». Коэффициенты и степени мономов результирующего полинома соответствуют ожидаемым математическим результатам.
- Обработка переполнения степеней:
  Тест **MultiplicationPolynomAnyThrow** проверил генерацию исключений при попытке получить степень переменной выше 9 (например, при перемножении мономов $x^9y^9z^9$ и $x^9y^9z^8$), что обеспечивает защиту от недопустимых значений степеней.

Анализ результатов:

Все тесты прошли успешно, за исключением сценариев, где намеренно передавались некорректные данные для проверки обработки ошибок. Результаты подтвердили, что:

- Односвязный список с фиктивной головой корректно управляет памятью и поддерживает порядок мономов.
- Полиномы корректно выполняют арифметические операции, включая сложение, вычитание и умножение, с соблюдением математических правил.
- Исключения и ограничения степеней переменных реализованы корректно, предотвращая недопустимые операции.

Проведённое тестирование подтверждает корректность работы всех ключевых компонентов программы и стабильность работы при обработке допустимых и недопустимых данных.

Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. **Реализация класса списков**: Разработан класс Forward_list_with_f_head, представляющий односвязный список с фиктивной головой, который обеспечивает эффективное хранение мономов полинома, упрощает управление памятью и гарантирует сохранение порядка элементов.
2. **Реализация класса полиномов**: Создан класс Polynomial, позволяющий выполнять основные арифметические операции над полиномами с ограничением на степень каждой переменной в диапазоне [0; 9]. Реализованы методы: сложение, вычитание, умножение на вещественную константу и умножение двух полиномов.
3. **Автоматизация упорядочивания**: Реализована логика сравнения и сортировки мономов по степени, которая учитывает приоритет каждой переменной. Это позволяет полиномам автоматически храниться в математически правильном порядке независимо от порядка ввода мономов или результатов арифметических операций. Пользователю не нужно вручную сортировать мономы – программа сама гарантирует корректное представление полинома и объединение подобных членов.
4. **Обработка исключений и проверка корректности данных**: Реализована проверка на допустимость коэффициентов и степеней мономов. В случае выхода за допустимые границы (например, степень <0 или >999) программа генерирует исключения, предотвращая некорректные вычисления.
5. **Тестирование**: С помощью фреймворка Google Test проведено всестороннее модульное тестирование. Проверены корректность вставки, копирования, присваивания, арифметических операций, умножения полиномов, а также обработка исключений при недопустимых данных. Тесты подтвердили правильность работы всех ключевых функций.
6. **Пользовательский интерфейс**: Реализовано консольное приложение, обеспечивающее удобный ввод полиномов, выбор операций и визуализацию результатов вычислений.

Таким образом, разработанная программа полностью соответствует поставленной задаче, корректно обрабатывает полиномы, обеспечивает надежность и удобство использования, а также подтверждена тестированием.

Литература

1. Сайт cppreference: https://en.cppreference.com/